

TLadder: QoE-Centric Video Ladder Optimization with Playback Feedback at Billion Scale

Zhuqi Li, He Liu, Shenglan Huang, Baojin Geng, Jack Chen, Jialin Chen, Liyang Sun, Qianyun Ma, Peizhe Liu, Junjun Zhao, Yiting Liao, Jamie Chen, Qian Ma, Qiang Ma, Feng Qian

ByteDance

Abstract

We operate one of the largest short video streaming platforms in the world, serving billions of global users every day. Similar to other platforms, we employ Adaptive Bitrate (ABR) streaming, which adaptively selects an appropriate representation from a group of video representations forming a *bitrate ladder*. In this paper, instead of developing yet another ABR algorithm, we optimize the bitrate ladder configuration, which constitutes the decision space of ABR, by developing a system called TLadder.

TLadder explicitly maximizes the quality-of-experience (QoE) that a ladder is expected to incur, through a principled optimization framework with polynomial complexity and provable optimality. TLadder's optimization jointly considers the video content dimension (*i.e.*, the bitrate-quality tradeoff of candidate representations) and the playback feedback dimension (*e.g.*, network condition, re-buffering time, and playback bitrate). TLadder also introduces a high-fidelity simulator that provides the optimizer with accurate QoE estimation. We evaluate TLadder via a large-scale A/B test across 0.24 billion users. TLadder achieves a **6.2–20%** reduction in rebuffering time, a **2.7–3.5** unit increase in visual quality (in an 85–100 unit range), and a **1–4%** reduction in CDN traffic cost, compared to our previous bitrate laddering approach. It also outperforms existing state-of-the-art approaches based on extensive trace-driven simulation.

CCS Concepts

• **Networks** → **Application layer protocols**; • **Information systems** → **Multimedia streaming**.

Keywords

Video Streaming, Video Transcoding, Bitrate Ladder, Adaptive Bitrate Streaming, Quality-of-Experience

ACM Reference Format:

Zhuqi Li, He Liu, Shenglan Huang, Baojin Geng, Jack Chen, Jialin Chen, Liyang Sun, Qianyun Ma, Peizhe Liu, Junjun Zhao, Yiting Liao, Jamie Chen, Qian Ma, Qiang Ma, Feng Qian. 2025. **TLadder: QoE-Centric Video Ladder Optimization with Playback Feedback at Billion Scale**. In *ACM SIGCOMM '25, Coimbra, Portugal*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGCOMM '25, Coimbra, Portugal

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1524-2/2025/09
<https://doi.org/10.1145/3718958.3750500>

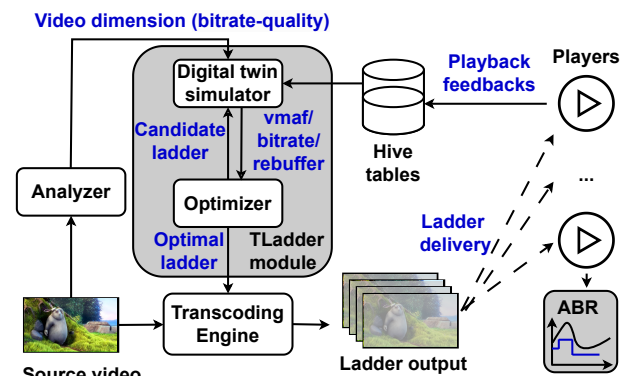


Figure 1: TLadder architecture.

2025 Conference (ACM SIGCOMM '25), September 8–11, 2025, Coimbra, Portugal. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3718958.3750500>

1 Introduction

Adaptive Bitrate (ABR) streaming has been extensively researched by the networking community over the past 15 years [1, 5, 7, 13, 14, 19, 25, 27, 30, 31, 34, 36, 42, 46]. In an ABR-capable video-on-demand (VoD) system, a video is encoded into multiple representations at varying bitrates/qualities; at runtime, the ABR logic, which can be embodied in handcrafted rules [14, 19], an optimization framework [36, 46], or a machine learning model [13, 30], adaptively selects an appropriate representation based on the network condition and/or the player's state (*e.g.*, the buffer occupancy level).

ABR requires a structured group of video representations (*i.e.*, quality levels) as input, forming a *bitrate ladder*. As the network condition improves (deteriorates), ABR's decision of increasing (decreasing) the quality can be thought as climbing up (down) the bitrate ladder. The ladder information is usually delivered to the client player as metadata before a streaming session starts.

Instead of developing yet another ABR algorithm, this paper focuses on the problem of *how to construct the ladder to maximize the quality-of-experience (QoE)*, as the ladder configuration plays an important role in making ABR effective: a ladder directly constrains the decision space available to the ABR; a poorly designed ladder with suboptimal bitrate-quality tradeoffs can thus restrict the client's ability to adapt effectively to changing network conditions. Under stable network conditions where ABR performs few bitrate switches, the ladder's impact on QoE is even more significant.

The challenges of ladder configuration lie in several aspects. **First**, it owns a large, discrete decision space as there may exist hundreds of candidate representations with diverse resolutions, codecs, and encoding parameters. The scalability becomes a particularly important problem when ladders are produced on a per-video basis. **Second**, the ladder design needs to collectively consider multiple candidate representations. This makes it fundamentally different from and orthogonal to the large body of work in encoding *individual* representations, such as codec upgrades [15, 16], per-shot encoding [3], and region of interest encoding [6, 10, 33]. **Third**, since a ladder is shared by many viewers, ladder decisions are made on the server/cloud side. Henceforth, the ladder system need to consider the *collective* QoE across viewers, as opposed to a single viewer's experience concerned by ABR. **Fourth**, a practical ladder solution is further constrained by the transcoding resources, *i.e.*, the limited CPU/GPU resources for encoding each representation in a ladder. Therefore, a naive solution of producing a dense ladder with an excessively large number of representations may not work for most videos in a production environment.

The bitrate ladder design receives far less attention compared to ABR. Traditional approaches (*e.g.*, by Apple [2] and Google [9]) employ static ladders not tailored to specific content characteristics, network environments, or users. Recent developments have made ladders aware of the video content [3] or usage context [35]. Nevertheless, these methods rely on coarse-grained and heuristic-driven rules, making it difficult for the resulting ladder to achieve the optimal QoE. Some other ladder systems [12, 17, 38] employ heavy-weight solvers or machine learning models, incurring significant scalability issues. Lastly, there are very few reports (only [4] and [3] to our knowledge) on the performance of commercial video ladder systems deployed *in the wild*.

Our study fills in the above gaps. We operate one of the largest short video streaming platforms in the world, serving billions of global users every day. This paper describes a novel video bitrate ladder system dubbed TLadder. Since its launch in 2024, TLadder has made transcoding (*i.e.*, ladder construction) decisions for more than 60 million videos watched by 0.8 billion users, with considerable QoE gains achieved. As shown in Fig. 1, for each uploaded source video, TLadder determines a QoE-wise optimal ladder. The transcoding engine then executes the decision by actually generating the video representations according to the ladder. The ladder information and the representations are then delivered to clients and consumed by the client-side ABR. TLadder is transparent to and requires no change to the ABR logic.

TLadder bears three key design aspects. First, unlike prior ladder designs [4, 35, 38], TLadder takes a holistic approach of *integrating the ideas of content- and context-aware encoding*. As shown in Fig. 1, TLadder takes two dimensions of input: the *video dimension*, which consists of predicted bitrate-quality tradeoff of candidate representations, and the *playback feedback dimension*, which includes key statistics (network condition, rebuffering, playback bitrate, *etc.*) collected from the production environment. TLadder then synthesizes both dimensions to derive the ladder configuration. This synergy makes TLadder robust to not only video content dynamics, but also fluctuating real-world conditions.

Second, TLadder develops a principled optimization framework to overcome the limitations of heuristic-driven rules [4, 35, 38].

As shown in Fig. 1, TLadder consists of an *optimizer* and a *digital twin simulator*. Given a list of candidate representations, the optimizer strategically enumerates candidate ladders and identifies the one with the maximal QoE. The simulator interacts with the optimizer by offering an interface for efficiently estimating the QoE for each candidate ladder through a data-driven “what-if” analysis. Specifically, the simulator takes into account the network speed distribution, the ladder selection distribution, and the rebuffering distribution, all obtained from the online data, to make the resulting QoE faithfully reflect ABR's decision and thus the watching experience in complex online environments. Decoupling the optimizer and simulator simplifies the system design, and allows two sub-modules to evolve independently.

Third, unlike the exponential complexity found in previous approaches [4, 38], TLadder can massively scale up to millions of videos per day through a polynomial-time optimization algorithm. Achieving the efficiency does not sacrifice the accuracy – the algorithm explores the full search space to obtain the provably optimal bitrate ladder. In addition, We address several practical issues, including dealing with client devices' heterogeneous decoding capabilities, seamlessly integrating TLadder with the transcoding pipeline, and developing system-level optimizations to speed up the algorithm execution.

We implemented TLadder and deployed it in our production system in 2024. We evaluate TLadder through a large-scale A/B experiment involving 0.24 billion users in two different regions. Compared to our previous ladder approach, which performs (coarse-grained) offline brute-force search for the optimal ladder, TLadder reduces rebuffering by **6.2–20%**, improves the perceptual quality (measured by VMAF [26]) by **2.7–3.5** units on average (in an 85-100 unit range), and reduces the CDN traffic cost by **1–4%**, under the same ABR algorithm. The benefits of TLadder are observed across diverse network conditions, content genres, and deployment regions. In addition, we conduct extensive trace-driven simulation to compare TLadder with existing approaches [4, 35, 38]. TLadder provides an average QoE gain of 2.0-4.3 units and 1.3-7.7 units (in a typical 70-90 unit range) in different regions.

To summarize, this work makes two-fold contributions: (1) the holistic design of a QoE-centric, content- and context-aware video ladder system, and (2) large-scale deployment and evaluation of TLadder in the wild. This work does not raise ethical issues. No customers' Personal Identifiable Information (PII) was collected or used in this work.

2 Background and Motivation

We provide sufficient background and motivate the design of TLadder.

2.1 Adaptive Bitrate (ABR) Streaming

Modern video streaming systems primarily rely on adaptive bitrate (ABR) algorithms to deliver content to heterogeneous client devices over dynamic networks. ABR algorithms measure or predict network speed and monitor client buffer occupancy to make real-time decisions about the bitrate of future video segments. For instance, if the ABR algorithm detects high available bandwidth and/or a sufficiently large buffer, it requests higher-quality segments to enhance the viewing experience [14, 19]. Conversely, if the algorithm

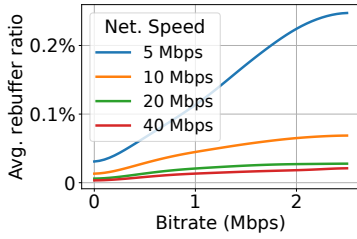


Figure 2: Average rebuffer ratio increases with the video bitrate at various network speed.

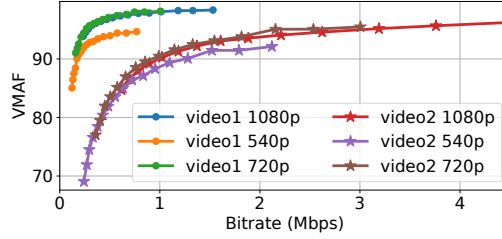


Figure 3: Bitrate-VMAF curve for two examples videos. Ladders are generated with different resolution and CRF.

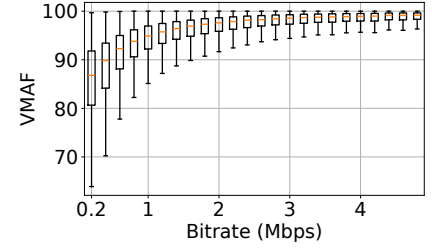


Figure 4: VMAF distribution at different video bitrate bucket across a 25,000 video datasets.

estimates reduced bandwidth or observes a dangerously low buffer level, it opts for lower-bitrate segments to avoid rebuffering. In addition to the above heuristic-driven ABR algorithms, recently proposed learning-based ABR approaches [13, 30] also follow similar high-level principles, despite their more complex, data-driven or machine-generated logic.

An ideal ABR logic adapts the video bitrate to the network condition and the player’s buffer status. However, due to the inherent difficulty in forecasting the (fluctuating) network condition, ABR may select an inappropriate bitrate level. In particular, if the network bandwidth cannot sustain the selected video bitrate, the video buffer may deplete and rebuffering (stalls) will occur. Rebuffering not only disrupts the viewing experience but also negatively impacts user engagement and satisfaction.

Rebuffering in the Wild. While it is well known that video bitrate plays a pivotal role in influencing rebuffering behavior, there are few reports on the quantitative relationship in large-scale production environments. We collected and analyzed 26 billion play sessions over a six-month period, sampling those with average network speeds between 5–40 Mbps. As shown in Fig. 2, we make two major observations. (i) Somewhat surprisingly, even when the available network bandwidth (5–40 Mbps) far exceeds the encoded video bitrate, a non-trivial rebuffer ratio persists. This is attributed to possibly high bandwidth fluctuation even when the (average) bandwidth is high. Note that a rebuffering ratio as low as 0.1% causes 1% reduction in users watch time, as observed from our metric data. (ii) Rebuffering increases nearly linearly with the video bitrate. This is not only due to higher bitrates increasing the risk of depleting the buffer, but also because once rebuffering occurs, high-bitrate content requires a longer recovery time. The above findings highlight *the prevalence of rebuffering* in both poor and good (in a statistical sense) network conditions.

2.2 Video Encoding & Visual Quality Primer

ABR requires the source video be encoded into a bitrate ladder consisting of multiple bitrate (quality) levels. This is usually achieved by varying the resolution and the *constant rate factor* (CRF) [45]. CRF controls the tradeoff between perceptual quality and bitrate: lower CRF values yield higher quality (and higher bitrates), whereas higher CRF values produce lower quality (and lower bitrates). Unlike fixed-bitrate encoding, CRF dynamically adjusts bits per frame to maintain a consistent visual quality. For example, encoding the

same source at CRF 18 versus CRF 23 can yield a notable difference in both the resulting bitrate and perceived quality.

At the same CRF, complex content with richer textures or higher motion typically results in higher bitrate compared to simpler content such as cartoons or static scenes. This is because higher-complexity content demands more bits to preserve critical details, whereas low-motion or low-detail frames need fewer bits to achieve a similar level of perceived quality. Such a variability underscores the importance of tailoring the bitrate ladder not only to the network distribution but also to the intrinsic complexity of each video.

As a concrete example, consider the RD (rate-distortion) curves in Fig. 3. An RD curve visualizes the encoder’s characteristics in controlling the bitrate and visual quality. Each mark in the RD curve corresponds to a CRF value (ranging from 20 to 36 in our experiment). We measure the visual quality using the Video Multi-Method Assessment Fusion (VMAF) score [26], a widely adopted perceptual video quality assessment method. As shown in Fig. 3, relatively “easy” content (video 1, an animation) can achieve high VMAF at lower bitrate, while “hard” content (video 2, sports footage) requires significantly higher bitrate to reach the same VMAF score. Fig. 4 further reveals that the VMAF variance across 25,000 videos at similar bitrates is considerable, underscoring the importance of content-aware approaches. Note that the VMAF score is capped at 100 and the VMAF tends to grow (statistically) as the bitrate increases. Therefore the VMAF variance decreases as the bitrate increases. Hence, due to the diverse RD-curve of different video content, a one-size-fits-all ladder cannot reliably optimize the tradeoff between quality and bitrate across diverse video types.

Fig. 3 also suggests that VMAF score exhibits diminishing returns as bitrate grows. This aligns with the well-known phenomenon that once video quality surpasses a certain threshold, human visual perception becomes progressively less sensitive to incremental improvements [26]. Consequently, striving for much higher bitrates offers only marginal perceptual benefits but substantially increases the likelihood of disruptive buffering. This tradeoff suggests a strong incentive to find a balanced ladder design that accounts for both user-perceived quality and buffer stability.

3 Design

The observations in §2.2 motivate the need for an enhanced ladder construction framework—one that intelligently integrates fine-grained content features with large-scale playback data (e.g., network speed traces, per-user rebuffering statistics) to deliver the best

overall Quality of Experience (QoE). TLadder takes a data-driven approach to address these issues, aiming to balance perceptual quality with robust performance under volatile network conditions.

TLadder includes an optimizer and a digital twin simulator. The optimizer runs efficiently to search for the best ladder that maximizes the QoE (§3.2). The digital twin simulator accurately predicts average bitrate, rebuffering durations, and overall video quality for each candidate ladder, which are used by the optimizer. (§3.3).

3.1 Problem Formulation

3.1.1 QoE Definition. Like video streaming systems, the goal of TLadder is to maximize the quality of experience (QoE) for users. A widely accepted QoE metric is a weighted combination of video quality, rebuffering penalty, and video quality switches [5, 18, 30, 46]. We follow this paradigm but introduce two modifications. First, for large-scale streaming platforms like ours, the data cost (*i.e.*, CDN traffic cost) is a crucial factor, which is billed by the amount of data transmitted by CDN and typically contributed as the largest cost factor for video streaming platforms. To accommodate this, we introduce a penalty term proportional to the average video bitrate. Second, we drop the video quality switch component mainly due to the short-video-serving nature of our business.¹ Therefore, we formulate the QoE as follows:

$$QoE = \text{vmaf} - \alpha \times \text{rebuffer} - \beta \times \text{bitrate} \quad (1)$$

where α and β are weights. Note that the QoE here is averaged across all the watch sessions rather than a single video play session since the bitrate ladder set is designed to serve millions of watching sessions. Also, TLadder's framework is generic, capable of working with different QoE metrics. We choose Eq. 1 since it meets our business requirement.

3.1.2 Optimization Object. We are given a list of candidate quality levels for a video, sorted by the quality levels' bitrates:

$$L_{in} = \{l_1, l_2, \dots, l_M\}.$$

Each quality level l_j is a unique bitrate-VMAF pair generated by specific encoding parameters (*i.e.*, resolution, CRF). We wish to select N quality levels from L_{in} to form a final output ladder,

$$L_{out} \subseteq L_{in}, \quad |L_{out}| = N,$$

so that the resulting QoE is maximized. Formally, we define the following optimization problem:

$$L_{out} = \arg \max_{\substack{S \subseteq L_{in} \\ |S|=N}} QoE(S). \quad (2)$$

3.2 Optimizer

The optimizer involves two major steps: *pre-filtering* and *Dynamic Programming (DP) optimization*. The optimizer is invoked on a per-video basis to product customized optimal ladder for every video.

¹Most videos served on our streaming platform are short videos. In these cases, quality switches are not as frequent as those during the playback of traditional long videos. In other words, our ABR algorithm primarily determines the bitrate level at the beginning of a video session.

3.2.1 Pre-filtering. The goal of the pre-filtering step is to reduce the computational complexity of the optimization process by pruning suboptimal quality levels from the input list L_{in} . This step is inspired by the concept of convex hulls [3] and ensures that only the most promising candidates are considered for further processing. Specifically, we evaluate each quality level in L_{in} based on its tradeoff between the visual quality (VMAF) and bitrate. A quality level l_i is considered suboptimal and removed from L_{in} if there exists another quality level l_j such that l_j has a higher VMAF and a lower bitrate than l_i . This filtering ensures that only those quality levels forming the Pareto front [40] in the VMAF-bitrate plane are retained. This pre-filtering simplifies the input space and improves the efficiency of the DP optimization step by reducing the number of candidate ladders to be evaluated. After pre-filtering, the remaining quality levels are monotonically increased in the VMAF-bitrate plane. We sorted quality levels in ascending order before passing it to the DP optimization.

3.2.2 DP optimization. We employ a dynamic programming (DP) framework to solve Eq. 2. Let $Q[i][j]$ be the highest attainable QoE when selecting i quality levels from the first j quality levels in (filtered and sorted) L_{in} . Also, we use $R_L[i][j]$ to record the actual set of quality levels (*i.e.*, the indices or references in L_{in}) that achieves the QoE in $Q[i][j]$. We establish the following DP formulation.

$$Q[i][j] = \begin{cases} \text{get_qoe}(R_L[i][j]) & i = j \\ \max_k (Q[i][j-1], \text{get_qoe}(R_L[i-1][k] \cup \{L_{in}[j]\})) & i < j \end{cases} \quad (3)$$

In Eq. 3, the trivial case is $i = j$ where the QoE can be directly calculated from $R_L[i][j] = \{L_{in}[1], L_{in}[2], \dots, L_{in}[i]\}$. The non-trivial case ($i < j$) examines two scenarios: (1) Not selecting the j -th ladder. In this scenario, the best i quality levels are selected from the first $j-1$ levels, and $R_L[i][j]$ is set to $R_L[i][j-1]$. (2) Selecting the j -th quality level. In this scenario, we first pick the best $i-1$ levels from the first k levels, and then add the j -th level; $R_L[i][j]$ is updated accordingly as $R_L[i-1][k'] \cup \{L_{in}[j]\}$ where k' is the best k that yields the highest QoE.

We now elaborate the DP algorithm. In Algorithm 1, we initialize all entries of $Q[i][j]$ to $-\infty$ so that any valid QoE value will be larger than the initial state. We similarly initialize the set $R_L[i][j]$ as empty for all i and j . We then populate the edge cases: as shown in Fig. 5(a), when we select exactly i ladders from the first i input ladders, we must pick all of them. Hence, $Q[i][i]$ is evaluated by calling `get_qoe` function on the first i ladders of L_{in} , and $R_L[i][i]$ holds those exact ladders. To account for the cases of selecting only one ladder, we allow $Q[1][j]$ (the QoE when choosing only one ladder out of j available) to be updated if a later ladder provides a better QoE when selected as a singleton set, as shown in Fig. 5(b). This ensures that our DP solution does not overlook any potentially higher-quality single ladder.

Following initialization, we proceed with a nested loop where i runs from 2 to N and j runs from $i+1$ to M as shown in Fig. 5(c). For each pair (i, j) , we examine two cases:

- (1) **Not selecting the j -th ladder.** In this case, $R_L[i][j] = R_L[i][j-1]$ and $Q[i][j] = Q[i][j-1]$.

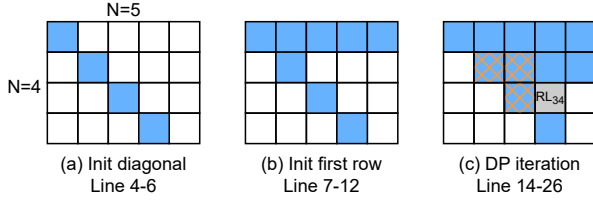


Figure 5: The execution steps for Algorithm 1. The algorithm first computes the QoE the diagonal elements and the first row of the DP matrix as initialization. Then to compute QoE of any element $R_L[i][j]$, it only need to consider if previous elements in the same row and the row immediately before.

- (2) **Selecting the j -th ladder.** We iterate over all possible k (from $i-1$ to $j-1$) where the previous $i-1$ ladders might have been chosen. We form a new candidate set by adding ladder j to $R_L[i-1][k]$. If the QoE of this new set, $\text{get_qoe}(L_{\text{new}})$, is higher than the current best $Q[i][j]$, we update $Q[i][j]$ and $R_L[i][j]$.

After processing all possible i and j , the algorithm tracks the best QoE found so far, denoted by q_{max} . The final output is the set of ladders L_{out} that yields q_{max} . In doing so, the algorithm efficiently explores all valid subsets of size N via dynamic programming, ensuring that the resulting solution is the optimal ladder configuration for maximizing QoE.

Correctness of DP. The key to proving the DP's optimality is to show its optimal substructure, i.e., if we select j as² the highest quality level in $R_L[i][j]$, which constitutes $Q[i][j]$, then the second-highest quality level k in $R_L[i][j]$ must also appear in the DP solution $R_L[i-1][k]$. To show this, suppose we can find a sub-ladder T that selects $i-1$ quality levels from the first k quality levels, and $k \notin T$ and $\text{get_qoe}(T) > \text{get_qoe}(R_L[i][j] \setminus \{j\})$; then it can be shown that $T \cup \{j\}$ will lead to a higher QoE than $Q[i][j]$, causing a contradiction. The detailed proof is in Appendix A.1.

3.3 Digital-Twin Simulator

The goal of the digital twin simulator is to estimate users' QoE for a given candidate ladder according to Eq. 1, i.e., realizing the $\text{get_qoe}(\cdot)$ function in Eq. 3. Given the difficulty of analytically modeling the QoE in complex production environments, the simulator takes a data-driven approach that leverages the massive performance data collected by our video-on-demand (VoD) metric platform. There are two key design requirements for the simulator. First, it should assess a "what-if" scenario where a candidate ladder is deployed online; second, the QoE should statistically reflect all users' experience when watching a given video, as opposed to a single user session's experience.

To meet the *first* requirement, without loss of generality, we decompose the what-if scenario into the three dimensions of perceived quality, bitrate, and rebuffering, according to Eq. 1. We then separately assess each dimension to simplify the overall process. For the *second* requirement, the simulator judiciously models various distributions including the network speed distribution, the quality

²For convenience, we use an index j to refer to the j -th quality level in L_{in} .

Algorithm 1: TLadder's DP Optimization Algorithm

Input: Input ladder list L_{in} (sorted from low to high quality), Output ladder count N

Output: Output ladder list L_{out} , Max QoE q_{max}

```

1  $M \leftarrow \text{length}(L_{\text{in}})$ ;
  //  $Q[i][j]$  records the max QoE if to choose  $i$ 
  // output ladders from the first  $j$  input ladders
  // and  $R_L[i][j]$  is the corresponding selected
  // ladder list.
2 Initialize  $Q[i][j] \leftarrow -\infty$  for  $i \in [1, N]$  and  $j \in [1, M]$ ;
3 Initialize  $R_L[i][j] \leftarrow \{\}$  for  $i \in [1, N]$  and  $j \in [1, M]$ ;
  // Initialize DP matrix
4 for  $i \leftarrow 1$  to  $N$  do
5    $R_L[i][i] \leftarrow \{L_{\text{in}}[1], L_{\text{in}}[2], \dots, L_{\text{in}}[i]\}$ ;
6    $Q[i][i] \leftarrow \text{get\_qoe}(R_L[i][i])$ ;
7 for  $j \leftarrow 2$  to  $M$  do
8    $R_L[1][j] \leftarrow R_L[1][j-1]$ ;
9    $Q[1][j] \leftarrow Q[1][j-1]$ ;
10  if  $\text{get\_qoe}(\{L_{\text{in}}[j]\}) > Q[1][j-1]$  then
11     $R_L[1][j] \leftarrow \{L_{\text{in}}[j]\}$ ;
12     $Q[1][j] \leftarrow \text{get\_qoe}(\{L_{\text{in}}[j]\})$ ;
  // Start DP iteration
13  $L_{\text{out}} \leftarrow R_L[1][M]$ ;  $q_{\text{max}} \leftarrow Q[1][M]$ ;
14 for  $i \leftarrow 2$  to  $N$  do
15   for  $j \leftarrow i+1$  to  $M$  do
16     // When not select the  $j$  th ladder,
17     //  $R_L[i][j]$  equals to  $R_L[i][j-1]$ .
18      $R_L[i][j] \leftarrow R_L[i][j-1]$ ;
19      $Q[i][j] \leftarrow Q[i][j-1]$ ;
20     for  $k \leftarrow i-1$  to  $j-1$  do
21       // Cases when select the  $j$  th ladder in
22       // the input list.
23        $L_{\text{new}} \leftarrow R_L[i-1][k] \cup \{L_{\text{in}}[j]\}$ ;
24        $q_{\text{new}} \leftarrow \text{get\_qoe}(L_{\text{new}})$ ;
25       if  $q_{\text{new}} > Q[i][j]$  then
26          $R_L[i][j] \leftarrow L_{\text{new}}$ ;  $Q[i][j] \leftarrow q_{\text{new}}$ ;
  // Record the best output ladder set.
27 if  $Q[i][M] > q_{\text{max}}$  then
28    $L_{\text{out}} \leftarrow R_L[i][M]$ ;  $q_{\text{max}} \leftarrow Q[i][M]$ ;
29 return  $L_{\text{out}}, q_{\text{max}}$ ;

```

levels' playback distribution, and the rebuffering distribution, to make the resulting QoE faithfully reflect ABR's decision and thus the watching experience in complex online environments.

We next describe the concrete approach we take. We first consider a simple scenario by assuming every playback session experiences the same average network speed t and is streamed at a fixed bitrate level r (the representation is thus also fixed). Let v be the corresponding representation's VMAF and $\mathcal{S}$ be the set of playback sessions in our dataset. The three QoE dimensions are:

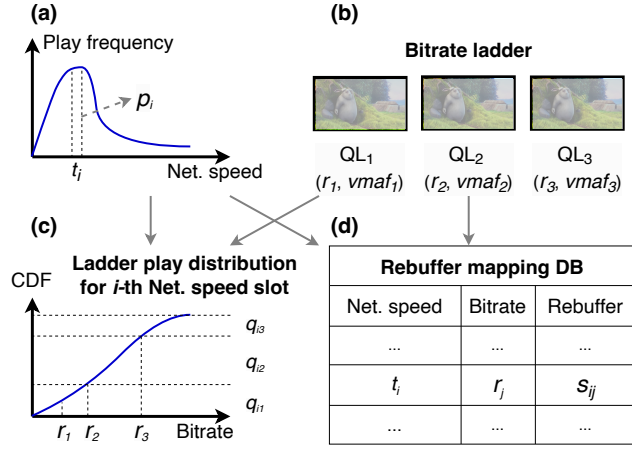


Figure 6: Key components for digital twin simulator. (a) is network speed distribution across all play sessions. (b) is the bitrate ladder from the optimizer, each quality level (QL) has bitrate and VMAF. (c) is the ladder play distribution collected from production metrics report. (d) is the rebuffer mapping database, which maps the (network speed, video bitrate) pair to the average rebuffering collected from metrics report.

$$\overline{\text{VMAF}} = v, \quad \overline{\text{Bitrate}} = r, \quad \overline{\text{Rebuf}} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \text{stall}_s.$$

Plugging them into the platform-wide QoE function (Eq. 1):

$$\text{QoE}(t, r) = v - \alpha \cdot \overline{\text{Rebuf}} - \beta \cdot r.$$

The above estimate reproduces the average experience observed in $\mathcal{S}$. Recall that all sessions in $\mathcal{S}$ belong to the same bucket of average network speed t and bitrate level r .

In reality, playback sessions span a wide range of network speeds and bitrates. We discretize the network speed into M slots $\{t_1, \dots, t_M\}$. Meanwhile, the candidate ladder $\mathcal{L}$ contains N representations whose VMAF values and bitrates are denoted as $\{v_1, \dots, v_N\}$ and $\{r_1, \dots, r_N\}$, respectively. Each (t_i, r_j) pair defines a bucket that aggregates sessions whose average network speed centers around t_i and whose streamed bitrate belongs to r_j . The average VMAF, rebuffering time, and bitrate of *all* sessions shaped by $\mathcal{L}$ is thus the sum of the corresponding metric across all the buckets, weighted by the buckets' occurrence (viewing) probabilities:

$$\overline{\text{VMAF}} = \sum_{i=1}^M p_i \sum_{j=1}^N q_{ij} v_j, \quad (4)$$

$$\overline{\text{Rebuf}} = \sum_{i=1}^M p_i \sum_{j=1}^N q_{ij} s_{ij}, \quad (5)$$

$$\overline{\text{Bitrate}} = \sum_{i=1}^M p_i \sum_{j=1}^N q_{ij} r_j. \quad (6)$$

In the above, p_i is the probability a session belongs to network speed slot t_i ; q_{ij} is the conditional probability that bitrate r_j is

chosen given t_i , per ABR's decision; s_{ij} is mean stall duration of the sessions falling in the (t_i, r_j) bucket.

To actually calculate Eq. 4 to 6, $\{r_j\}$ and $\{v_j\}$ are parameters of the candidate ladder; p_i , q_{ij} , and s_{ij} are derived from the "big data", i.e., our metric data platform, as shown in Fig. 6. Specifically, $\{p_i\}$ can be directly queried from the data platform. For $\{q_{ij}\}$, we first generate the bitrate distributions $\{R_i\}$ for each network speed slot i . For each R_i , we then calculate $\{q_{ij}\}$ using the candidate ladder's bitrate distribution $\{r_j\}$. For $\{s_{ij}\}$, we build a rebuffering database that maintains for each (network speed slot, video bitrate) pair the average rebuffering duration collected from the data platform. More dimensions (e.g., player's buffer level) could be added to increase the accuracy of rebuffering time estimation. Also, $\{p_i\}$, $\{q_{ij}\}$, and $\{s_{ij}\}$ are refreshed daily to make the simulator adaptive to changes of our production environment and customers' network conditions.

Finally, the simulator returns the calculated QoE for the candidate ladder $\mathcal{L}$ to the optimizer:

$$\text{QoE}(\mathcal{L}) = \overline{\text{VMAF}} - \alpha \cdot \overline{\text{Rebuf}} - \beta \cdot \overline{\text{Bitrate}} \quad (7)$$

3.4 Other Practical Issues

3.4.1 Speeding Up Algorithm Execution. TLadder uses two techniques to further improve its computational efficiency, facilitating its global-scale deployment.

First, within the optimizer, the most expensive operation is the repeated invocation of the `get_qoe(·)` function in Eq. 3 (i.e., the digital twin simulator). We therefore employ a status cache that maps a hash of any given candidate bitrate ladder to its corresponding QoE result. This cache eliminates redundant calls to the simulator whenever the same ladder configuration is encountered.

Second, within the digital twin simulator, TLadder utilizes the highly-optimized matrix multiplication operation to compute the average VMAF, rebuffer, and bitrate. Eq. 4 to 6 can be equivalently written in matrix forms as follows:

$$\widehat{\text{vmaf}} = \mathbf{P}^T \mathbf{Q} \mathbf{V}, \quad (8)$$

$$\widehat{\text{rebuffer}} = \mathbf{P}^T (\mathbf{Q} \circ \mathbf{S}) \mathbf{1}, \quad (9)$$

$$\widehat{\text{bitrate}} = \mathbf{P}^T \mathbf{Q} \mathbf{R}, \quad (10)$$

where $\mathbf{P} \in \mathbb{R}^M$ is the column vector whose i -th component is p_i . $\mathbf{Q} \in \mathbb{R}^{M \times N}$ is the matrix with entries q_{ij} . $\mathbf{V} \in \mathbb{R}^N$ is the vector whose j -th component is vmaf_j . $\mathbf{S} \in \mathbb{R}^{M \times N}$ is the matrix with entries s_{ij} . $\mathbf{R} \in \mathbb{R}^N$ is the vector whose j -th component is r_j . $\mathbf{1} \in \mathbb{R}^N$ is the column vector of all ones (size N). $(\mathbf{Q} \circ \mathbf{S})$ denotes the elementwise (Hadamard) product of $\mathbf{Q}$ and $\mathbf{S}$. With matrix optimization, the overall computation can speed up by 10.3× based on our offline performance test.

3.4.2 Handling Devices' Heterogeneous Decoding Capabilities. Our streaming platform serves billions of users spanning a wide range of devices with varying decoding capabilities. Certain low-end devices may suffer long first-frame delays, exhibit black screens, or even crash when rendering content at certain resolutions or encoded by specific codecs. To address these issues, we maintain a dynamic database of device-specific decoding blocklists, so that problematic codecs or resolutions will not be distributed to affected devices. This mechanism is outside the scope of TLadder.

Rank	Quality Levels	QoE	Cost
1	QL #5	60	10
2	QL #5, QL #17	70	25
3	QL #5, QL #17, QL #2	77	35
4	QL #5, QL #17, QL #2, QL #12	82	47

Table 1: Example for the interface between TLadder and transcoding scheduling system.

In addition, *within* TLadder, for each quality level in a candidate ladder, TLadder penalizes the expected QoE by the fraction of user sessions that cannot decode it. Specifically, suppose the decode failure ratio is θ for a certain resolution (or codec). Then in the digital twin simulator, we calculate the QoE twice. One for the original candidate ladder and the other for the candidate ladder with the all quality levels in this resolution (or codec) removed. Then the two QoE results are combined with the weights of $1 - \theta$ and θ . This ensures that the ladder optimization process for each video takes into account practical device compatibility constraints. Note that both mechanisms help avoid widespread playback errors while still striving for high QoE across all compatible devices, but they are complementary to each other. In particular, performing penalization within TLadder reduces wasted transcoding, *i.e.*, representations that are transcoded but not consumed by clients.

3.4.3 Working with Popularity-based Transcoding Scheduler. Video transcoding requires considerable computing resources. To efficiently utilize them, the transcoding engine (Fig. 1) uses a popularity-based scheduling approach. Its basic idea is to predict each uploaded video’s popularity (*i.e.*, its estimated number of viewers); for high-popular (low-popular) videos, the transcoding engine transcodes more (fewer) quality levels. The scheduler is similar to [37] and its design details are beyond the scope of this paper. Note that the same transcoding scheduler is used in both the control and experiment groups in our A/B experiment (§5.2).

To work with this popularity-based scheduler, TLadder provides multiple *ranked subsets* of a full ladder as exemplified in Table 1, where a higher (lower) rank has more (fewer) quality levels. This allows the transcoding engine to flexibly select the appropriate rank (quality level subset) based on the predicted popularity tier. To produce the ranked subsets, TLadder employs a greedy algorithm as a post-processing step. Specifically, the algorithm takes the full ladder generated by the optimizer (§3.2) as the input; then from the full ladder, it iteratively produces each rank: to make rank i , it adds one quality level in the full ladder yielding the highest marginal QoE gain relative to the transcoding cost compared to rank $i - 1$ (*i.e.*, $(QoE_i - QoE_{i-1}) / (Cost_i - Cost_{i-1})$). The QoE gain is evaluated by the `get_qoe()` function in Eq. 3, and the transcoding cost is estimated by the average CPU time/video length for the corresponding (resolution, CRF) pair based on past transcoding records. This process continues until all quality levels in the full bitrate ladder are added. A ladder with N levels will produce N ranked subsets to be consumed by the transcoding scheduler.

4 Implementation

Our implementation consists of four major components: (1) the core TLadder module (the optimizer and digital twin simulator), (2) the playback data feedback loop, (3) the RD-curve prediction module, and (4) candidate representation selection. In total, the engineering implementation includes around 12,000 lines of code.

The Core TLadder Module. As the centerpiece of our transcoding pipeline, TLadder is implemented as a stateless Python function that integrates both the optimizer and the digital twin simulator. Inside TLadder, the optimizer interacts with the digital twin simulator through direct function calls and outputs the optimal ladder decision.

Interaction with the Transcoding Engine. The transcoding engine performs transcoding tasks under two circumstances: (1) when a video is uploaded by a user, a small number of default representations are immediately transcoded (without involving TLadder) to ensure basic user experiences; (2) as the video becomes popular, the transcoding engine consults TLadder to obtain the ladder configuration (see §3.4.3). Once TLadder makes its decision, it pushes the decided encoding parameters to a message queue, where transcoding workers pull them to carry out the actual transcoding. This design allows TLadder to handle concurrent transcoding requests distributively and efficiently at scale.

The Playback Data Feedback Loop provides the digital twin simulator with the playback feedback data for QoE estimation (§3.3). Our platform collects playback-related metrics for every streaming session in JSON format. This collected information consists of performance indicators such as network speed, rebuffering events, and average bitrate. Each metrics report is streamed to a dedicated pipeline that ingests and aggregates the data into Apache Hive databases. Building upon this infrastructure, TLadder executes a recurring daily batch job that queries the Hive databases to retrieve updated playback status information. This makes TLadder adaptive to changes of our production environment and customers’ network conditions. The retrieved metrics are then fed into the digital twin simulator to guide the optimizer to find the optimal ladder.

RD-curve Prediction. Recall from §3.3 that the digital twin simulator also requires data in the video content dimension, *i.e.*, RD-curves that capture the bitrate-quality tradeoff of candidate quality levels. To calculate an RD-curve, the traditional approach is to perform a frame-by-frame comparison between the source and the encoded output, but that requires fully transcoding all representations, incurring prohibitively high computing cost. Instead, TLadder runs on *predicted* VMAF and bitrate through our internal video quality assessment tool. The tool includes two components: feature extraction and model training/prediction. First, the tool quickly scans the input video through a specially designed lightweight encoder to extract encoding features. There are three types of features: video bit stream features, quality features, and video content features. Obtained from the encoded bit stream, the bit stream features include I/B/P frames’ ratios, number of bits, and Quantization Parameters (QP), as well as statistics of encoding mode decisions. The quality features are collected at the frame level, including Peak Signal-to-Noise Ratio (PSNR) [11] and Structural Similarity Index Measure (SSIM) [44], plus the overall VMAF of the

encoded video stream (note this VMAF is produced by our light-weight encoder and differs from the to-be-predicted VMAFs). The content features refers to the Spatial Information (SI) and Temporal Information (TI) [29], which reflects the spatial texture complexity and the amount of temporal motion. All the features are concatenated as one input vector encompassing a dimension of around 100. Next, for each codec and resolution, we use the above features (together with the candidate representation's CRF value) and the ground truth bitrate/VMAF value (as the label) to train a gradient boosting regressor. The two prediction targets, bitrate and VMAF, are trained in separate models. The training occurs offline, yielding ~ 20 models for different (codec, resolution, prediction target) tuples. The models are then used at runtime for bitrate/VMAF inference. We construct a dataset of 50k diverse videos for training and cross validation. We achieve a Mean Absolute Error (MAE) of $\sim 5\%$ for bitrate prediction and a MAE of 0.75 for VMAF prediction.

Candidate Representation Selection. We select candidate quality levels using a range of Constant Rate Factor (CRF) settings, combined with multiple spatial resolutions. Specifically, the CRF values range from 20 to 36, incremented in steps of 0.2, and the resolutions include 360p, 480p, 540p, 720p, and 1080p. In total, this yields 405 candidate representations. The number of quality levels in the full ladder ($N = |L_{\text{out}}|$ in Eq. 2) is chosen based on total transcoding resource budget. Typically N is around 5-12 for different regions in our online environment. The above parameters are empirically chosen and fully configurable.

5 Evaluation

Our evaluation focuses on three key questions regarding TLadder's performance, scalability, and adaptability:

- (1) How does TLadder compare to state-of-the-art bitrate ladder systems in terms of QoE? We observe that TLadder consistently outperforms existing approaches in both online A/B tests and trace-driven simulations, achieving VMAF improvements of 2.7–3.5 units (in an 85-100 unit range), rebuffer reductions of 6.2–20%, and CDN cost savings of 1–4% (§5.2).
- (2) Is TLadder computationally scalable to billion-user deployments? We show that TLadder generates ladder decisions within 2 seconds on a single-core CPU and consumes less than 0.2% of total transcoding resources, enabling real-time operation at large scales (§5.3).
- (3) How well does TLadder adapt to online variations, such as ABR policies and noisy inputs? Our experiments demonstrate that TLadder maintains high QoE under both conservative and aggressive ABR modes, and remains robust to errors injected into metrics like bitrate, VMAF, and network speed, with only a minimal QoE loss of 0.0008–0.178 units compared with the QoE decided by TLadder using ground truth data (§5.4).

5.1 Evaluation Setup

Methodology. We evaluate TLadder against baseline systems using two approaches: *large-scale A/B experiments* and *trace-driven simulation*.

	Alpha	Beta
Developed Country	2375	0
Developing Country	6000	0.0065

Table 2: Parameters for two experimental regions.

A/B experiments. In the A/B experiment, videos are transcoded using different bitrate ladder systems. The resulting bitrate ladders are delivered to separate user groups, which are randomly and equally split using our A/B experimentation platform. The transcoding cost is approximately 50,000 dollars per group per experiment.

Trace-driven simulation. We utilize the simulator described in §3.3 for trace-driven simulation. In the online environment, TLadder relies on data feedback from the previous day to make ladder decisions. Following the same paradigm in offline simulations, we generate the ladders with the data on day T and then evaluate the results with the data from day $T+1$. This approach ensures the offline evaluations follow the online operational workflow.

Evaluation Regions. We present the evaluation results in two different regions in the world: the **developed region** and the **developing region**. The developed region includes all developed countries with a daily active users (DAU) of 0.11 billion. The developing region includes all developing countries with a daily active users of 0.13 billion. The QoE parameters are different in the two regions due to different network distribution, device capability, and users' preference. The parameters used in the evaluation are listed in Table 2. The methodology to select the parameters is as follows. We first use offline simulation to explore the tradeoffs incurred by the parameters as illustrated in Fig. 11 and Fig. 12. We then narrow down the parameter space by shortlisting those achieving a Pareto improvement *i.e.*, higher VMAF, lower rebuffering, and lower CDN cost compared with the baseline. Lastly, given the shortlisted candidate parameters, we empirically use several pilot A/B experiments to finalize the parameter selection to be used in Eq. 1. Note that the same parameters apply to both A/B experiments and simulation.

Baselines. In the experiment, we conduct comprehensive comparisons between TLadder and four previous representative baseline systems from both research and industry.

- **VLadder** has been our previous online bitrate ladder system used in production for four years. At a very high level, it also searches for the QoE-wise best ladder as TLadder does, and its QoE function is the same as Eq. 1. However, VLadder differs from TLadder in several aspects. (1) It performs brute-force search for the optimal ladder. Due to the exponential nature, the search is performed at a very coarse granularity, and on a per-region instead of a per-video basis as TLadder does. (2) To get the rebuffering and bitrate components in the QoE function, VLadder employs a static look-up database generated offline through trace-driven simulation, therefore lacking the ability of dynamically adapting to the online playback feedback. (3) In each region, VLadder conducts the above brute-force search to produce several candidate ladders, which then compete through pilot A/B experiments for the winner to be eventually launched in production.

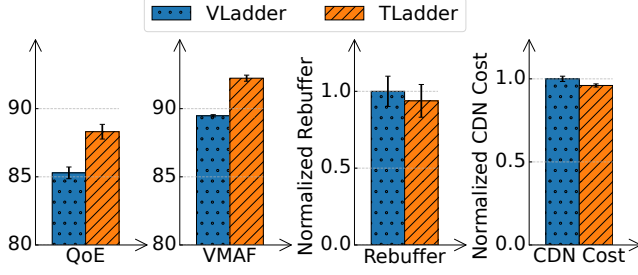


Figure 7: A/B experiment on developed region (with 0.11 billions DAU).

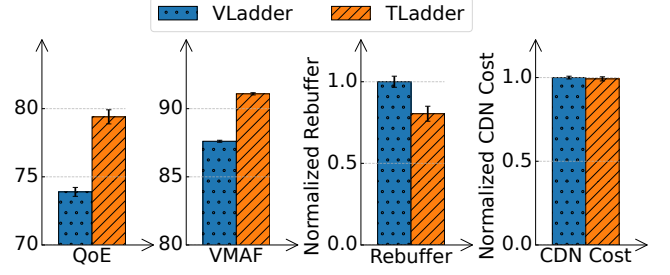


Figure 8: A/B experiment on developing region (with 0.13 billions DAU).

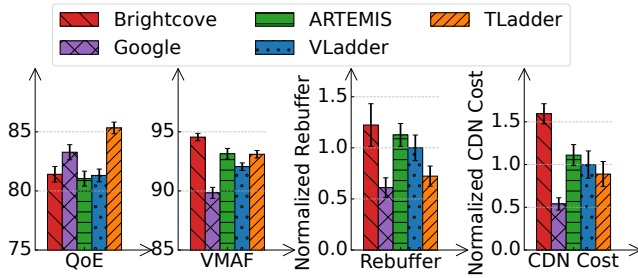


Figure 9: Simulated experiment on developed region (with 0.11 billions DAU).

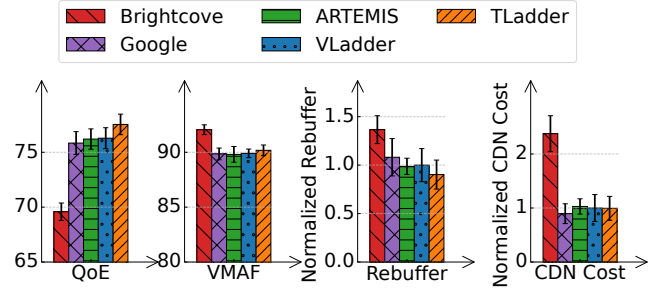


Figure 10: Simulated experiment on developing region (with 0.13 billions DAU).

- **ARTEMIS** [38] is a state-of-the-art bitrate ladder system for live streaming. We implement the algorithm based on the paper’s description. To make it suitable for the VoD setting, our implementation make its parameter α tunable rather than determined on-the-fly in a try-and-error manner.
- The **Google** Ladder [4] is the laddering system that has been deployed in large scale at Google. We implement the algorithm based on the paper.
- **Brightcove** [35] is a bitrate ladder system that joint considers the video content and network speed distribution. We implement the algorithm based on the description and parameters provided by the paper.

For ARTEMIS, Brightcove, and the Google Ladder, please refer to §6 for their details.

5.2 End-to-End Result

A/B Experiment. Fig. 7 and 8 show the results from two-week A/B experiments conducted in both the developed region (with 0.11 billion DAU) and the developing region (with 0.13 billion DAU). Each bar shows the daily average, and the error bars indicate variance across different days. We examine key QoE metrics (overall QoE, VMAF, rebuffer rate, and CDN cost) to evaluate TLadder’s performance against VLadder. For developed region, (Fig. 7) TLadder raises the average QoE from 85.3 to 88.3 and boosts VMAF by 2.7 units (from 89.5 to 92.2). Note that most online VMAF values typically range between 85 and 100 for 1 Mbps (*i.e.*, our average online bitrate) based on our measurement (see Fig. 4), so an 2.7-unit

improvement is considerable. At the same time, TLadder achieves a 6.2% reduction in rebuffering, while also lowering CDN cost by 4%. For developing region (Fig. 8), where rebuffering is more severe, TLadder boosts the QoE by over 5.5 units (from 73.9 to 79.4), while increasing VMAF by 3.5 units (from 87.6 to 91.1). Notably, it cuts rebuffering by nearly 20% and slightly reduces CDN cost by 1%. Overall, these findings confirm that by incorporating real playback feedback, TLadder provides substantial improvements in *both* user-perceived video quality and playback smoothness, which typically pose a tradeoff, across diverse production environments.

Result for trace-driven simulation. To further assess TLadder, we conduct large-scale trace-driven simulations for both the developed and developing regions, each matching the same populations as in our A/B experiments. In each region, we randomly sample 20,000 videos in production and transcode the videos with the bitrate ladders from TLadder and the four baselines. For each simulation, we measure four key metrics—QoE, VMAF, rebuffering ratio, and CDN cost—across all videos. The bar plots (Fig. 9 and 10) depict the mean values for each metric, with error bars showing the standard deviation over the sampled dataset.

For the developed region, (Fig. 9) TLadder achieves the highest QoE (85.3 on average, 2.0–4.3 units increase from the baselines) among all five approaches and delivers a top-tier VMAF score (93.1) with comparatively low rebuffering. Although Brightcove slightly outperforms TLadder in VMAF, it does so at a higher CDN cost. VLadder and ARTEMIS maintain moderate VMAF levels but suffer from higher rebuffering rates, which significantly lower their QoE. Meanwhile, Google is over conservative in the VMAF despite the

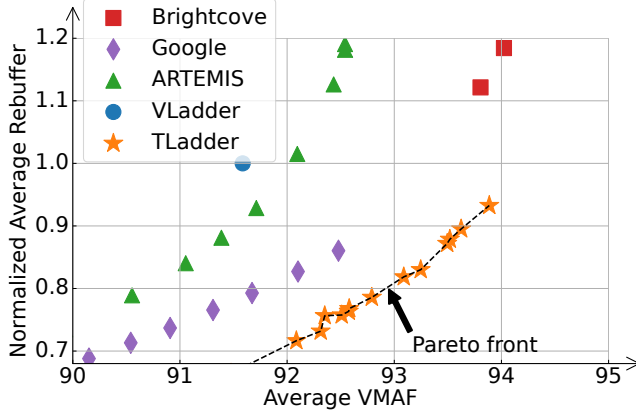


Figure 11: Selection space for different algorithms in the developed region.

Name	Input Size		
	19	85	405
Brightcove	0.0158	0.0158	0.0158
ARTEMIS	2.164	-	-
Google	14.973	148.691	-
VLadder	0.0023	0.0023	0.0023
TLadder	0.163	0.728	2.044

Table 3: Average computation time (in seconds) for a single video laddering decision on a single-core Intel i7-13700K CPU. ‘-’ indicates the algorithm did not complete within 3,600 seconds.

lowest rebuffering and CDN cost. Overall, TLadder strikes a balanced tradeoff between visual quality, playback smoothness, and CDN cost.

For the developing region (Fig. 10), rebuffering is more severe due to more challenging network conditions. Although TLadder’s improvement in video quality (VMAF) over VLadder, ARTEMIS, and Google is modest (roughly 0.3 units), TLadder substantially reduces rebuffering rates, resulting in a notable boost to QoE (1.3 – 7.7 units improvement). In contrast, Brightcove’s more aggressive approach to boosting video quality leads to a significantly higher rebuffering and CDN cost, henceforth yielding the lowest QoE among the evaluated methods.

TLadder is at the Pareto front. To further explore the efficiency of different approaches in navigating the tradeoff between video quality and playback smoothness, we plot the *average VMAF* (x-axis) against the *normalized average rebuffer rate* (y-axis) in Fig. 11 and Fig. 12. Each marker style corresponds to a specific bitrate ladder system, and each dot is generated by a unique parameter set for a given algorithm. As shown, TLadder (orange stars) yields an optimal tradeoff between video quality and playback smoothness. It forms the *Pareto front* [40], *i.e.*, the set of points for which there is no strictly better solution on both metrics. Other approaches are less effective in balancing this critical trade-off. This is attributed to

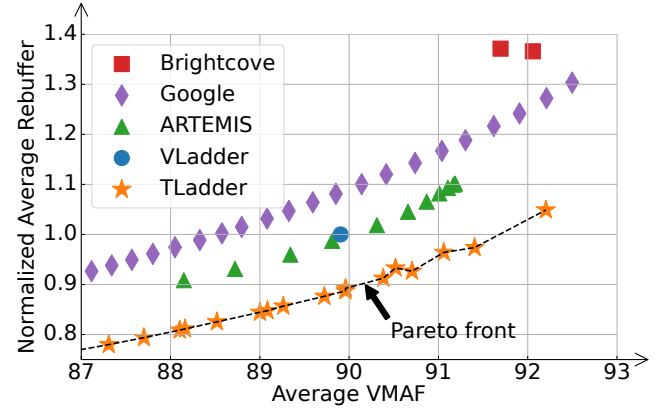


Figure 12: Selection space for different algorithms in the developing region.

TLadder’s fine-grained strategy that adapts to both content variations and users’ playback feedback. This leads to ladder choices that balance trade-offs between video quality and playback smoothness.

5.3 Computational scalability

Offline Cost Evaluation. We next examine the computational overhead of constructing the bitrate ladder. We summarize the results for Brightcove, TLadder, ARTEMIS, VLadder, and Google in Table 3. We examine the cost under three input sizes of candidate quality levels: (1) **19**, matching the configuration used in ARTEMIS; (2) **85**, sampling five resolutions with CRF ranging from 20 to 36 in increments of 1.0; and (3) **405**, sampling the same five resolutions with CRF increments of 0.2 (the minimal CRF resolution for our codec). For Brightcove and VLadder, the computational cost does not increase with the scale of the input (yet they considerably underperform TLadder in QoE, as reported in §5.2). For the largest input size of 405, ARTEMIS and Google either time out or become prohibitively slow (fail to finish in 3,600 seconds). In contrast, TLadder completes that in **2.044** seconds. For smaller inputs (19 and 85), TLadder incurs only **0.163** seconds and **0.728** seconds, respectively, also significantly faster than Google and ARTEMIS. This demonstrates TLadder’s high computational efficiency, making it suitable for real-time ladder construction at scale.

Online Resource Usage. Fig. 13 plots the cumulative distribution function (CDF) of TLadder’s computation overhead relative to the cost of transcoding those decided ladders based on our online transcoding logging. Specifically, the x-axis represents the ratio of TLadder’s processing cost to the full transcoding cost; the y-axis indicates the percentage of videos that fall at or below a given ratio. We observe that TLadder’s overhead is consistently small, with most cases (80th percentile) falling below 0.2% of the transcoding cost. This low overhead makes TLadder efficient for large-scale online deployment.

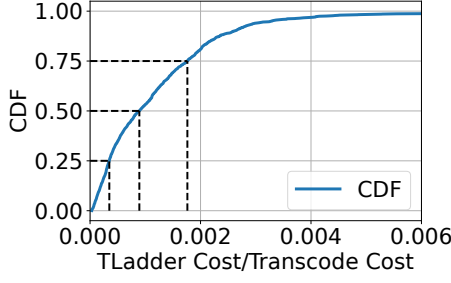


Figure 13: TLadder's computation cost vs transcoding cost.

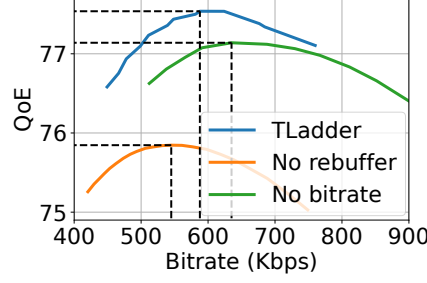


Figure 14: The impact of modeling rebuffering and bitrate in TLadder.

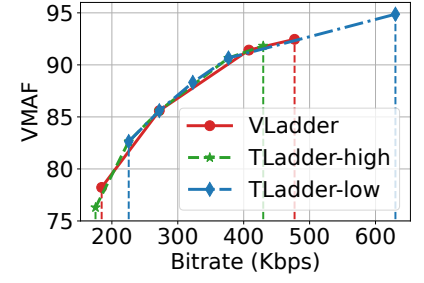


Figure 15: A case study for how rebuffer impacts TLadder's decision.

5.4 Micro Benchmarks

Impacts of Rebuffering and Bitrate Modeling. In designing TLadder, we explicitly model both rebuffering and bitrate penalties into our optimization objective. To isolate each penalty's effect, we examine two alternatives that exclude one penalty at a time: **No rebuffer**, which ignores rebuffering costs, and **No bitrate**, which ignores bitrate penalties. Fig. 14 plots the resulting QoE values against the simulated average playback bitrate for each approach; different points on each curve are generated by varying parameters to produce diverse bitrate ladder configurations. We find that TLadder consistently achieves higher QoE compared to either variant that omits one of the penalties. Notably, rebuffering penalty makes a large contribution in QoE as the orange curve is lower than the green one.

Adapting to ABR Algorithms. Fig. 16 compares multiple approaches under two ABR modes: Conservative and Aggressive. In the conservative mode, the ABR algorithm is tuned to prefer playback smoothness, whereas in the aggressive mode, the ABR algorithm shifts towards a higher video quality with the risk of rebuffering. In both settings, TLadder consistently achieves the highest QoE among all solutions. Interestingly, TLadder adapts its output ladder based on the ABR's behavior. When the ABR is conservative, TLadder generates more aggressive ladders (i.e., emphasizing higher-quality encodings) because it anticipates that the ABR will under-utilize available bandwidth. Conversely, when the ABR is aggressive, TLadder adopts slightly more conservative ladder configurations to avoid excessive rebuffering and cost. This dynamic self-adjustment, thanks to TLadder's capability of taking the playback feedback, allows TLadder to maintain a high QoE regardless of the underlying ABR policy.

Robustness analysis. To evaluate TLadder's tolerance to prediction inaccuracies, we inject noises into four ground-truth metrics: bitrate, VMAF, network speed, and rebuffering. Fig. 17 shows the impact of each error type, with noise levels varied along the x-axis. We use the noisy data to generate TLadder's bitrate ladder, and then measure the QoE on the unperturbed ground truth. VLadder serves as the baseline for performance comparison.

As illustrated by the dashed vertical lines in each sub-figure, typical online prediction errors fall well within the tested noise range. The average QoE lost owing to prediction noises for bitrate, VMAF, network speed, and rebuffer ratio is **0.065**, **0.178**, **0.038**,

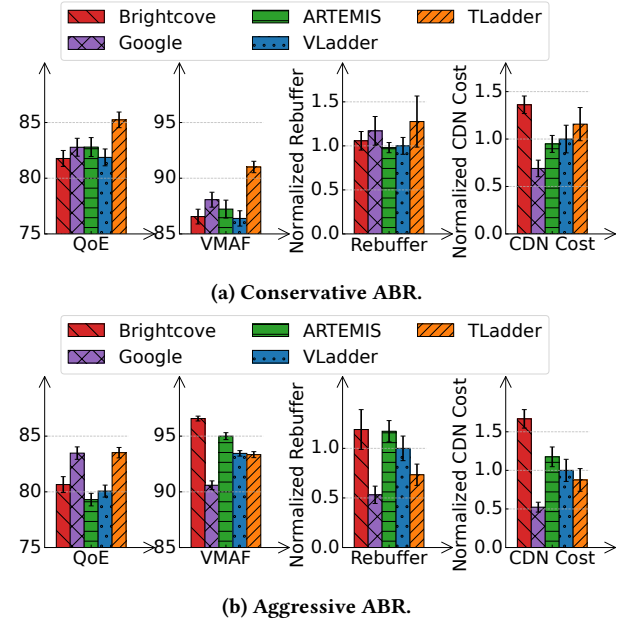


Figure 16: TLadder can adapt to different ABR mode.

and **0.0008**, respectively. In these real-world regimes, TLadder consistently outperforms VLadder with only minimal QoE degradation compared with the TLadder's decision with ground truth input. Overall, these findings confirm TLadder's resilience to real-world variability and highlight its ability to maintain high QoE despite imperfect predictions.

5.5 A Case Study

Fig. 15 presents a case study illustrating how rebuffering influences TLadder's decisions. The x-axis shows the bitrate of each quality level, and the y-axis represents the corresponding VMAF. Each curve corresponds to a distinct bitrate ladder, with individual markers indicating specific quality levels in the ladder. We plot TLadder-produced ladders in both high (TLadder-high) and low (TLadder-low) rebuffering scenarios, alongside the reference ladder from VLadder.

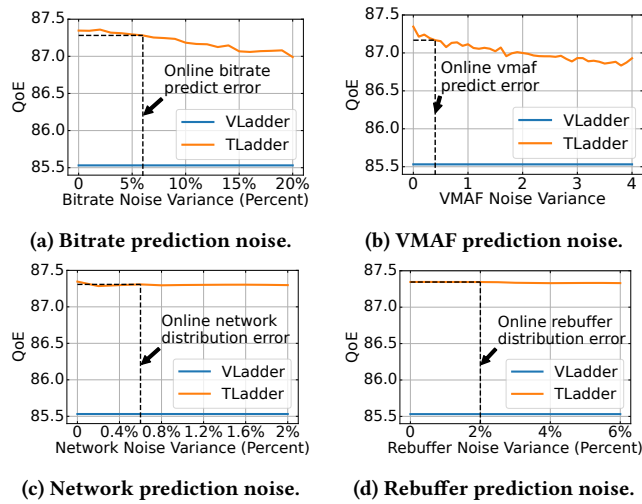


Figure 17: Robustness study under varying prediction errors. Each subfigure shows TLadder and VLadder QoE as we incrementally inject noise into a different metric. The dashed lines mark typical online error levels.

From the figure, it is clear that increased rebuffering drives TLadder-high to select lower-bitrate (and therefore lower-quality) profiles to mitigate stalls. Conversely, when rebuffering is minimal, TLadder-low shifts its selections toward higher-quality, higher-bitrate profiles. In this manner, TLadder dynamically adapts to varying levels of rebuffering feedback, striking a desired balance between visual quality and playback smoothness.

6 Related Work

Video bitrate systems originate from static configurations with fixed bitrates [2, 9] and have evolved into more advanced approaches including content-aware approaches, context-aware approaches, and real-time adaptation approaches.

Content-aware approaches. Most existing content-aware methods can be broadly classified into two categories. The first category is *bitrate ladder estimation* [21, 23, 32, 41], which predicts the resulting video quality (e.g., VMAF) from codec parameters (e.g., resolution, CRF) without fully transcoding all ladder candidates. The second category is *bitrate ladder construction* [3, 8, 22, 28], which first builds rate-distortion (RD) curves using the estimated bitrate-quality relationship and then applies heuristic rules to select the final ladder set. In contrast to the above approaches, we leverage playback statistics to guide ladder selection, thus achieving a finer-grained tradeoff among video quality, rebuffering, and cost in streaming systems.

Context-Aware Approaches A number of approaches integrate user context—such as network distributions [4, 12, 17, 35], quitting ratios [24], or viewport usage in 360-degree video [43]—to refine bitrate ladder decisions in VoD.

Specifically, Brightcove’s approach [35] classifies video content into three models (easy, medium, hard) and network conditions into

two models (easy, hard) to determine the optimal bitrate ladder. Because it relies on just a few discrete categories, this coarse-grained method often produces suboptimal ladders. In contrast, TLadder employs a fine-grained, dynamic programming-based strategy that adapts to both content variations and playback feedback without being constrained by rigid categories. Consequently, TLadder provides significantly more precise ladder choices.

Google’s approach [4] sets a fixed quality target and minimizes bitrate under that constraint, without accounting for rebuffering penalties. For simple content, meeting the fixed target may underutilize available bandwidth and miss opportunities to improve quality. For more complex content, achieving the same target can require excessively high bitrates, leading to frequent rebuffering. In contrast, TLadder directly considers both video quality and rebuffering, ensuring a balanced trade-off between quality and smooth playback across all types of content.

Deep learning-based methods [12, 17] use neural networks to jointly analyze content and network distributions, but their high computational overhead can be prohibitive for large-scale deployments—sometimes even surpassing the cost of transcoding itself. TLadder overcomes these scalability challenges by employing a polynomial-time DP optimization strategy and an efficient digital twin simulator.

Real-time adaptation approaches Bitrate ladder systems for live streaming such as ARTEMIS [38] and LALISA [39] employ trial-and-error methodologies to dynamically adapt bitrate ladders during live sessions. Similar to the design of TLadder, ARTEMIS also follows the idea of using playback feedback to optimize ladder configurations. But the difference is that ARTEMIS uses a mixed-integer linear programming (MILP) solver to find the best ladder combination. ARTEMIS has two main drawbacks compared to TLadder. First, the MILP solver is computationally expensive, which restricts the ARTEMIS’s solution space, leading to suboptimal outcomes since optimal points might often lie outside the convex hull of the rate-distortion curves. Second, ARTEMIS has scalability issues. It can practically support only hundreds of concurrent players, which makes it struggle for large-scale deployment.

In contrast, TLadder uses an efficient DP algorithm instead of a MILP solver, allowing it to explore the codec’s full parameter space and find truly optimal ladder configurations. TLadder also scales effectively to billions of users, making it better suited for large-scale streaming platforms.

7 Discussions

Applicability to Long Videos. While our production deployment mostly targets short videos (*i.e.*, using the same ladder for a given video entirely), TLadder generalizes naturally to long videos (*i.e.*, using different ladders in different parts of a video). Specifically, a common industry practice to handle a long video is to partition it into short segments, and encode each segment with its own bitrate ladder—a procedure known as per-shot encoding [20]. In this setting, TLadder can be applied segment by segment (or more practically, every k segments), delivering similar benefits to those for short videos. One caveat is that a quality switch penalty across segments may need to be added to the QoE formulation for long videos when applying TLadder.

Work with Diverse ABR Strategies. In our production environment, we use a heuristic-based ABR algorithm that adjusts the bitrate based on the buffer occupancy level [14], among other factors. Nevertheless, TLadder’s design renders it agnostic to the client ABR as its optimization loop consumes solely video content and usage context information, *i.e.*, video bitrate/quality, observed playback bitrate, and rebuffering time, rather than any internal logic of ABR. Consequently, when engineers swap in a new ABR strategy, the empirical joint distribution of the above observations shifts spontaneously to reflect the new ABR strategy. TLadder will then re-ingest the update through a recurring daily batch job that queries the Hive databases to retrieve updated playback status information. This leads to updated ladders that maximize the expected QoE under the new operation regime, without requiring code change or hyper-parameter tuning.

Re-transcoding (Updaing) Video Ladders is supported by our transcoding pipeline. However, in our current deployment, a ladder is only transcoded once per video. This design is attributed to two reasons. First, our transcoding resources are limited. Second, videos in our platform receive most of views within three days of their creation, similar to other video streaming services [37]. Within such a short time, the distributions of playback feedback are usually stable. This renders the benefits of re-transcoding video ladders marginal.

8 Lessons Learned and Future Work

We share several lessons learned from developing TLadder and describe the future work. First, our results indicate that bitrate ladder optimization is as important as the ABR algorithm. While ABR

has become relatively mature, much fewer efforts from the networking/system community have been made on improving bitrate ladders. This is in part due to the inter-disciplinary nature of ladder optimization (*e.g.*, accurate RD-curve prediction is a research topic in multimedia) and perhaps more importantly, a lack of large-scale datasets that are essential for evaluating ladder approaches. We advocate both aspects.

Second, we demonstrate at scale that using playback feedback data in addition to the video content information can benefit ladder decisions. While enriching the feedback data (*e.g.*, incorporating a player’s buffer state) could further help, a more promising direction is to change the feedback from one-way to two-way, *i.e.*, performing co-optimization between ABR and the ladder algorithm. We are actively working in this direction.

Third, for video streaming in production environments, cost is an important factor to consider – not only the network bandwidth cost but also transcoding cost and the cost for making transcoding decisions, as shown in this work. In our current pipeline, it is the job of the transcoding scheduler (outside TLadder, see §3.4) to prioritize transcoding tasks based on the available transcoding resources and videos’ popularity. Jointly optimizing ladder construction (determining which representations to transcode) and transcoding scheduling (deciding when to transcode a given video’s representation) is another promising direction we plan to pursue.

Acknowledgments

We thank our shepherd Mohamed Hefeeda and the anonymous reviewers for their suggestions and feedback.

References

- [1] Zahaib Akhtar, Yun Seong Nam, Ramesh Govindan, Sanjay Rao, Jessica Chen, Ethan Katz-Bassett, Bruno Ribeiro, Jibin Zhan, and Hui Zhang. 2018. Oboe: Auto-tuning video ABR algorithms to network conditions. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. 44–58.
- [2] Apple Developer. [n. d.]. HLS Authoring Specification for Apple Devices. <https://developer.apple.com/documentation/http-live-streaming/hls-authoring-specification-for-apple-devices>. ([n. d.]).
- [3] Netflix Technology Blog. 2015. Per-Title Encode Optimization. <https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2>. (2015).
- [4] Chao Chen, Yao-Chung Lin, Steve Benting, and Anil Kokaram. 2018. Optimized transcoding for large scale adaptive streaming using playback statistics. In *2018 25th IEEE International Conference on Image Processing (ICIP)*. IEEE, 3269–3273.
- [5] Tianyu Chen, Yiheng Lin, Nicolas Christianson, Zahaib Akhtar, Sharath Dharmaji, Mohammad Hajiesmaili, Adam Wiernan, and Ramesh K Sitaraman. 2024. SODA: An adaptive bitrate controller for consistent high-quality video streaming. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 613–644.
- [6] Zhenzhong Chen, Junwei Han, and King Ng Ngan. 2006. Dynamic bit allocation for multiple video object coding. *IEEE Transactions on Multimedia* 8, 6 (2006), 1117–1124.
- [7] Federico Chiariotti, Stefano D’Aronco, Laura Toni, and Pascal Frossard. 2016. Online learning adaptation strategy for DASH clients. In *Proceedings of the 7th International Conference on Multimedia Systems*. 1–12.
- [8] Jan De Cock, Zhi Li, Megha Manohara, and Anne Aaron. 2016. Complexity-based consistent-quality encoding in the cloud. In *2016 IEEE International Conference on Image Processing (ICIP)*. IEEE, 1484–1488.
- [9] Google Support. [n. d.]. Select live encoder settings, bitrate and resolution. <https://support.google.com/youtube/answer/2853702>. ([n. d.]).
- [10] Sunhyoung Han and Nuno Vasconcelos. 2008. Object-based regions of interest for image compression. In *Data Compression Conference (dcc 2008)*. IEEE, 132–141.
- [11] Alain Hore and Djemel Ziou. 2010. Image quality metrics: PSNR vs. SSIM. In *2010 20th international conference on pattern recognition*. IEEE, 2366–2369.
- [12] Tianchi Huang, Rui-Xiao Zhang, and Lifeng Sun. 2021. Deep reinforced bitrate ladders for adaptive video streaming. In *Proceedings of the 31st ACM Workshop on Network and Operating Systems Support for Digital Audio and Video*. 66–73.
- [13] Tianchi Huang, Chao Zhou, Rui-Xiao Zhang, Chenglei Wu, Xin Yao, and Lifeng Sun. 2019. Comyco: Quality-aware adaptive video streaming via imitation learning. In *Proceedings of the 27th ACM International Conference on Multimedia*. 429–437.
- [14] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. 2014. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 187–198.
- [15] ITU-T and ISO/IEC JTC 1/SC 29. 2013. High Efficiency Video Coding. <https://www.itu.int/rec/T-REC-H.265>. (2013). ITU-T Recommendation H.265 and ISO/IEC 23008-2.
- [16] ITU-T and ISO/IEC JTC 1/SC 29. 2020. Versatile Video Coding. <https://www.itu.int/rec/T-REC-H.266>. (2020). ITU-T Recommendation H.266 and ISO/IEC 23090-3.
- [17] Lianchen Jia, Chao Zhou, Tianchi Huang, Chaoyang Li, and Lifeng Sun. 2023. RDladder: Resolution-Duration Ladder for VBR-encoded Videos via Imitation Learning. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [18] Junchen Jiang, Vyas Sekar, Henry Milner, Davis Shepherd, Ion Stoica, and Hui Zhang. 2016. {CFA}: A practical prediction system for video {QoE} optimization. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. 137–150.
- [19] Junchen Jiang, Vyas Sekar, and Hui Zhang. 2012. Improving fairness, efficiency, and stability in http-based adaptive video streaming with festive. In *Proceedings of the 8th international conference on Emerging networking experiments and technologies*. 97–108.
- [20] Ioannis Katsavounidis et al. 2018. Dynamic optimizer-a perceptual video encoding optimization framework. *The NETFLIX tech blog* 4 (2018).
- [21] Angeliki V Katsenou, Joel Sole, and David R Bull. 2019. Content-gnostic bitrate ladder prediction for adaptive video streaming. In *2019 Picture Coding Symposium (PCS)*. IEEE, 1–5.
- [22] Angeliki V Katsenou, Joel Sole, and David R Bull. 2021. Efficient bitrate ladder construction for content-optimized adaptive video streaming. *IEEE Open Journal of Signal Processing* 2 (2021), 496–511.
- [23] Angeliki V Katsenou, Fan Zhang, Kyle Swanson, Mariana Afonso, Joel Sole, and David R Bull. 2021. VMAF-based bitrate ladder estimation for adaptive streaming. In *2021 Picture Coding Symposium (PCS)*. IEEE, 1–5.
- [24] Pierre Lebreton and Kazuhisa Yamagishi. 2023. Quitting ratio-based bitrate ladder selection mechanism for adaptive bitrate video streaming. *IEEE Transactions on Multimedia* 25 (2023), 8418–8431.
- [25] Wenzhong Li, Xiang Li, Yeting Xu, Yi Yang, and Sanglu Lu. 2023. Metaabr: A meta-learning approach on adaptive bitrate selection for video streaming. *IEEE Transactions on Mobile Computing* 23, 3 (2023), 2422–2437.
- [26] Zhi Li, Christos Bampis, Julie Novak, Anne Aaron, Kyle Swanson, Anush Moorthy, and JD Cock. 2018. VMAF: The journey continues. *Netflix Technology Blog* 25, 1 (2018).
- [27] Zhuqi Li, Yaxiong Xie, Ravi Netravali, and Kyle Jamieson. 2023. Dashlet: Taming swipe uncertainty for robust short video streaming. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 1583–1599.
- [28] Melissa Licciardello, Lukas Humbel, Fabian Rohr, Maximilian Grüner, and Ankit Singla. 2022. Prepare your video for streaming with segue. *arXiv preprint arXiv:2202.09112* (2022).
- [29] Suiyi Ling, Yoann Baveye, Patrick Le Callet, Jim Skinner, and Ioannis Katsavounidis. 2020. Towards perceptually-optimized compression of user generated content (ugc): Prediction of ugc rate-distortion category. In *2020 IEEE international conference on multimedia and expo (ICME)*. IEEE, 1–6.
- [30] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with pensieve. In *Proceedings of the conference of the ACM special interest group on data communication*. 197–210.
- [31] Zili Meng, Yining Guo, Yixin Shen, Jing Chen, Chao Zhou, Minhu Wang, Jia Zhang, Mingwei Xu, Chen Sun, and Hongxin Hu. 2021. Practically deploying heavyweight adaptive bitrate algorithms with teacher-student learning. *IEEE/ACM Transactions on Networking* 29, 2 (2021), 723–736.
- [32] Fatemeh Nasiri, Wassim Hamidouche, Luce Morin, Nicolas Dholand, and Jean-Yves Aubié. 2022. Multi-preset video encoder bitrate ladder prediction. In *Proceedings of the 2nd International Workshop on Design, Deployment, and Evaluation of Network-Assisted Video Streaming*. 8–13.
- [33] Yura Perugachi-Diaz, Guillaume Sautière, Davide Abati, Yang Yang, Amirhossein Habibi, and Taco S Cohen. 2022. Region-of-interest based neural video compression. *arXiv preprint arXiv:2203.01978* (2022).
- [34] Yanyuan Qin, Ruofan Jin, Shuai Hao, Krishna R Pattipati, Feng Qian, Subhabrata Sen, Chaoqun Yue, and Bing Wang. 2019. A control theoretic approach to ABR video streaming: A fresh look at PID-based rate adaptation. *IEEE Transactions on Mobile Computing* 19, 11 (2019), 2505–2519.
- [35] Yuriy A Reznik, Karl O Lillevold, Abhijith Jagannath, Justin Greer, and Jon Corley. 2018. Optimal design of encoding profiles for ABR streaming. In *Proceedings of the 23rd Packet Video Workshop*. 43–47.
- [36] Kevin Spiteri, Rahul Urgaonkar, and Ramesh K Sitaraman. 2020. BOLA: Near-optimal bitrate adaptation for online videos. *IEEE/ACM transactions on networking* 28, 4 (2020), 1698–1711.
- [37] Linpeng Tang, Qi Huang, Amit Puntambekar, Ymir Vigfusson, Wyatt Lloyd, and Kai Li. 2017. Popularity prediction of facebook videos for higher quality streaming. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. 111–123.
- [38] Farzad Tashtarian, Abdelhak Bentaleb, Hadi Amirpour, Sergey Gorinsky, Junchen Jiang, Hermann Hellwagner, and Christian Timmerer. 2024. {ARTEMIS}: Adaptive Bitrate Ladder Optimization for Live Video Streaming. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 591–611.
- [39] Farzad Tashtarian, Abdelhak Bentaleb, Hadi Amirpour, Babak Taraghi, Christian Timmerer, Hermann Hellwagner, and Roger Zimmermann. 2023. Lalisa: Adaptive bitrate ladder optimization in http-based adaptive live streaming. In *NOMS 2023-IEEE/IFIP Network Operations and Management Symposium*. IEEE.
- [40] CFI Teams. [n. d.]. Pareto Efficiency. ([n. d.]). <https://corporatefinanceinstitute.com/resources/economics/pareto-efficiency/>
- [41] Ahmed Telili, Wassim Hamidouche, Sid Ahmed Fezza, and Luce Morin. 2023. Efficient Per-Shot Transformer-Based Bitrate Ladder Prediction for Adaptive Video Streaming. In *2023 IEEE International Conference on Image Processing (ICIP)*. IEEE, 1835–1839.
- [42] Guibin Tian and Yong Liu. 2012. Towards agile and smooth video adaptation in dynamic HTTP streaming. In *Proceedings of the 8th international conference on Emerging networking experiments and technologies*. 109–120.
- [43] Laura Toni and Pascal Frossard. 2017. Optimal representations for adaptive streaming in interactive multiview video systems. *IEEE Transactions on Multimedia* 19, 12 (2017), 2775–2787.
- [44] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing* 13, 4 (2004), 600–612.
- [45] Ffmpeg Wiki. [n. d.]. H.264 Video Encoding Guide - Constant Rate Factor (CRF). ([n. d.]). <https://trac.ffmpeg.org/wiki/Encode/H.264#crf>
- [46] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. 2015. A control-theoretic approach for dynamic adaptive video streaming over HTTP. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 325–338.

A Appendices

Appendices are supporting material that has not been peer-reviewed.

A.1 Proof for DP Algorithm Optimality

In this section, we prove that our dynamic programming (DP) algorithm produces an optimal set of quality levels under two key assumptions about how the adaptive bitrate (ABR) logic selects quality levels and how the resulting QoE changes with increasing bitrates.

A.1.1 Assumptions.

A1. Highest feasible quality level selection. For each play session, the ABR algorithm attempts to play at the highest quality level that is feasible given the current network/buffer status/bitrate budget. If that quality level is not feasible, the algorithm falls back to the next-lower quality level, continuing in this manner until a feasible quality level is found. Therefore, whether a play session uses the *highest* quality level is independent of the specific availability of lower quality levels.

A2. Monotonic QoE property. Suppose there is a set of play sessions that currently use quality level j . If there exists a strictly higher-quality level j^+ (i.e., $j^+ > j$) such that switching these sessions to j^+ leads to a higher overall QoE, then for any lower-quality level $j^- < j$, the QoE attained by using j^- is strictly less than that attained by using j . In other words, once a higher quality level can yield a better QoE, any lower quality level must yield a worse QoE relative to j .

These assumptions reflect realistic properties of adaptive streaming:

- **(AA1.)** enforces a practical ABR strategy that always tries to maximize quality when feasible.
- **(AA2.)** aligns with the fact that, as long as a higher bitrate is still within a range that does not excessively increase re-buffering or penalty terms, the QoE gain from higher quality dominates. Before hitting a “break-even” point (where the additional penalty outweighs the gain), quality increases yield monotonically increasing QoE.

Under these assumptions, we now present a key lemma (Lemma A.1) that provides an important “monotonic increment” property of the QoE function. This lemma will be used in the proof of Theorem A.2 to show that selecting a highest quality level j naturally forces the choice of the second-largest quality level from among the previously optimal solutions.

LEMMA A.1. *Let S and T be two ladder sets with*

$$\max(S) = s, \quad \max(T) = t, \quad \text{and} \quad s > t,$$

where $\max(S)$ denotes the highest quality level in the ladder set S . Suppose we add a strictly higher-quality level j (where $j > s > t$) to both sets, forming

$$S \cup \{j\} \quad \text{and} \quad T \cup \{j\}.$$

If

$$\begin{aligned} \text{get_qoe}(S \cup \{j\}) &> \text{get_qoe}(S), \\ \text{get_qoe}(S \cup \{j\}) &> \text{get_qoe}(T \cup \{j\}), \end{aligned}$$

then

$$\text{get_qoe}(S) > \text{get_qoe}(T).$$

PROOF. Define:

$$\begin{aligned} \Delta_S &= \text{get_qoe}(S \cup \{j\}) - \text{get_qoe}(S), \\ \Delta_T &= \text{get_qoe}(T \cup \{j\}) - \text{get_qoe}(T). \end{aligned}$$

Since j is the new highest quality level for both $S \cup \{j\}$ and $T \cup \{j\}$, the subset of play sessions that can actually play at level j is the same for both sets. Call this set of sessions ϕ .

- For S , each session in ϕ upgrades its quality level from s to j . - For T , each session in ϕ upgrades its quality level from t to j .

Since $t < s < j$, upgrading from t to j increases QoE more than upgrading from s to j . Thus:

$$\Delta_S < \Delta_T.$$

Rewriting the given inequalities using Δ_S and Δ_T :

$$\text{get_qoe}(S \cup \{j\}) - \Delta_S > \text{get_qoe}(T \cup \{j\}) - \Delta_T$$

implies

$$\text{get_qoe}(S) > \text{get_qoe}(T),$$

which concludes the proof. $\square$

THEOREM A.2. *In the dynamic programming construction, if j is chosen as the maximum in $R_L[i][j]$ (i.e., the top ladder for an i -ladder solution), then the second-largest ladder must come from the DP solution $R_L[i-1][k]$.*

PROOF. We want to prove that if $j = \max(R_L[i][j])$, the second-largest ladder

$$k = \max(R_L[i][j] \setminus \{j\})$$

must be the top ladder from an optimal $(i-1)$ -ladder solution $R_L[i-1][k]$.

Let

$$S = R_L[i][j] \setminus \{j\} \quad \text{and} \quad T$$

be any ladder set chosen from the first k inputs, but which does *not* include k . By Lemma A.1, since

$$\text{get_qoe}(S \cup \{j\}) > \text{get_qoe}(T \cup \{j\}),$$

we know

$$\text{get_qoe}(S) > \text{get_qoe}(T).$$

Hence, k must appear in the best $(i-1)$ -ladder configuration (i.e., it must be part of $R_L[i-1][k]$ that maximizes $Q[i-1][k]$). In other words, if j is selected as the highest ladder for the i -ladder set, the choice of the next-highest ladder necessarily follows from the optimal $(i-1)$ -ladder solution. $\square$

Interpretation. Intuitively, the lemma shows a *monotonic increment* property of QoE when adding a strictly higher ladder. This ensures that if adding ladder j to a set S outperforms adding ladder j to a set T , then even without ladder j , set S must still outperform T . In the DP algorithm, whenever we choose a new highest quality j , the choice of the second-highest ladder is dictated by the best solution among the remaining $i-1$ ladders. Taken together, the lemma and theorem guarantee that our DP construction yields the optimal combination of ladders under Assumptions A1. and A2..